

ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: Phát triển ứng dụng IoT - Nhóm 1

MÃ HỌC PHẦN: 2025-2026.2.TIN4024.001

Phát triển hệ thống đèn LED thông minh RGB với ESP32

NHÓM: 19

Giảng Viên :Võ Việt Dũng

HUẾ THÁNG 4 - 2025

I. DANH MỤC CÁC TỪ VIẾT TẮT

I. DANH MỤC CÁC TỪ VIẾT TẮT

AC (Alternating Current): Dòng điện xoay chiều.

API (Application Programming Interface): Giao diện lập trình ứng dụng.

BLE (Bluetooth Low Energy): Công nghệ Bluetooth tiết kiệm năng lượng.

CSS (Cascading Style Sheets): Ngôn ngữ định dạng giao diện trang web.

DC (Direct Current): Dòng điện một chiều.

DHCP (Dynamic Host Configuration Protocol): Giao thức cấu hình động máy chủ.

GPIO (General-Purpose Input/Output): Chân tín hiệu đầu vào/đầu ra đa dụng.

HTML (HyperText Markup Language): Ngôn ngữ đánh dấu siêu văn bản.

HTTP (Hypertext Transfer Protocol): Giao thức truyền tải siêu văn bản.

IC (Integrated Circuit): Mạch tích hợp (Vi mạch).

IDE (Integrated Development Environment): Môi trường phát triển tích hợp.

JSON (JavaScript Object Notation): Định dạng dữ liệu văn bản nhẹ, dễ đọc và ghi.

LED (Light Emitting Diode): Đi-ốt phát quang.

MQTT (Message Queuing Telemetry Transport): Giao thức nhắn tin dành cho các thiết bị IoT.

PCB (Printed Circuit Board): Bảng mạch in.

PWM (Pulse Width Modulation): Điều chế độ rộng xung.

RGB (Red - Green - Blue): Không gian màu cơ bản (Đỏ - Xanh lá - Xanh dương).

SPIFFS (Serial Peripheral Interface Flash File System): Hệ thống tập tin bộ nhớ flash giao tiếp ngoại vi nối tiếp.

SSID (Service Set Identifier): Định danh mạng (Tên mạng Wi-Fi).

UI/UX (User Interface / User Experience): Giao diện người dùng / Trải nghiệm người dùng.

MỤC LỤC

DANH MỤC CÁC TỪ VIẾT TẮT	[2]
MỤC LỤC	[3]

PHẦN MỞ ĐẦU	[4]
--------------------------	------------

3.1. Tổng quan đề tài	[4]
3.2. Mục tiêu nghiên cứu	[5]
3.3. Đối tượng và phạm vi nghiên cứu	[5]
3.4. Phương pháp nghiên cứu	[6]

PHẦN NỘI DUNG	[6]
----------------------------	------------

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG NHÀ THÔNG MINH VÀ CÔNG NGHỆ CHIẾU SÁNG IOT

[6]

1.1. Khái niệm và kiến trúc cơ bản của Internet of Things (IoT)	[6]
1.2. Ứng dụng của IoT trong hệ thống chiếu sáng thông minh	[7]
1.3. Tổng quan về các giao thức mạng sử dụng trong hệ thống	[7]

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VỀ PHẦN CỨNG HỆ THỐNG

[8]

2.1. Vi điều khiển ESP32 và sức mạnh trong các ứng dụng IoT	[8]
2.1.1. Kiến trúc phần cứng và thông số kỹ thuật	[8]
2.1.2. Khả năng kết nối không dây	[9]
2.1.3. Sơ đồ chân (Pinout) và giao tiếp ngoại vi	[9]
2.1.4. So sánh ESP32 và ESP8266	[9]
2.2. Công nghệ LED RGB Neopixel WS2812	[10]
2.2.1. Cấu tạo và nguyên lý hiển thị màu sắc	[10]
2.2.2. Giao thức truyền dữ liệu một dây (One-wire data transmission)	[10]
2.2.3. Yêu cầu khắt khe về thời gian (Timing Requirements)	[11]
2.3. Các linh kiện phụ trợ bảo vệ hệ thống	[12]
2.3.1. Bộ nguồn tổ ong 5V công suất lớn	[12]
2.3.2. Điện trở bảo vệ chân dữ liệu (330Ω - 470Ω)	[12]
2.3.3. Tụ điện lọc nguồn (Bypass Capacitor - $1000\mu\text{F}/6.3\text{V}$)	[13]

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

[13]

3.1. Sơ đồ khối tổng thể của hệ thống	[14]
3.2. Sơ đồ nguyên lý kết nối phần cứng (Schematic)	[15]
3.3. Thiết kế phần mềm	[15]
3.3.1. Lưu đồ thuật toán của hệ thống	[16]
3.3.2. Cấu trúc giao diện Web Control	[18]
3.3.3. Cấu hình nền tảng đám mây (Sinric Pro) để giao tiếp	[19]
3.3.4. Các bước liên kết tài khoản IoT với ứng dụng Google Home	[20]
3.5. Phân tích và thiết kế mã nguồn điều khiển (Firmware)	[21]
3.5.1. Khai báo thư viện và cấu hình hệ thống	[21]
3.5.2. Quản lý trạng thái và Cấu trúc dữ liệu	[21]
3.5.3. Xây dựng các hàm hiệu ứng (Utility Functions)	[21]
3.5.4. Khởi tạo Web Server và RESTful API	[22]
3.5.5. Vòng lặp chính (Main Loop)	[22]

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	[22]
4.1. Hình ảnh thực tế mô hình đã lắp ráp	[22]
4.2. Quá trình kiểm thử giao diện Web Control	[23]
4.3. Quá trình kiểm thử bằng giọng nói (Google Assistant)	[24]
4.4. Đánh giá ưu và nhược điểm của hệ thống	[25]
4.4.1. Ưu điểm	[25]
4.4.2. Nhược điểm và hạn chế	[25]
5. PHẦN KẾT LUẬN VÀ KIẾN NGHỊ (HƯỚNG PHÁT TRIỂN)	[26]
5.1. Kết luận chung	[26]
5.2. Kiến nghị và Hướng phát triển tương lai	[27]
5.2.1. Phát triển phần cứng chuyên nghiệp (Thiết kế PCB)	[27]
5.2.2. Bổ sung tính năng đồng bộ ánh sáng theo âm nhạc (Music Sync)	[27]
5.2.3. Khắc phục sự phụ thuộc mạng và nâng cấp bảo mật	[28]
5.2.4. Cập nhật giao thức Matter	[28]
TÀI LIỆU THAM KHẢO	[28]
7. PHỤ LỤC	[29]

3. PHẦN MỞ ĐẦU

3.1. Tổng quan đề tài

Trong kỷ nguyên của cuộc Cách mạng công nghiệp 4.0, Internet vạn vật (Internet of Things - IoT) đã và đang tạo ra những bước đột phá mạnh mẽ trong mọi lĩnh vực của đời sống xã hội. Không chỉ dừng lại ở các ứng dụng công nghiệp, IoT đã len lỏi vào không gian sinh hoạt thường nhật thông qua khái niệm "Nhà thông minh" (Smart Home). Theo đó, mọi thiết bị điện tử từ điều hòa, tivi, đến hệ thống an ninh đều có khả năng kết nối mạng, thu thập dữ liệu và giao tiếp với con người [1, 12]. Trong hệ sinh thái này, chiếu sáng thông minh đóng một vai trò cốt lõi không chỉ vì mục đích cung cấp ánh sáng đơn thuần mà còn giúp tiết kiệm năng lượng, bảo vệ sức khỏe thị lực và cá nhân hóa không gian sống, tạo ra các hiệu ứng tâm lý tích cực cho người sử dụng.

Tuy nhiên, các hệ thống đèn thông minh thương mại trên thị trường hiện nay thường có giá thành khá cao và hoạt động trong các hệ sinh thái đóng (closed-

ecosystem), gây khó khăn cho việc tùy biến hoặc tích hợp chung với các nền tảng khác. Chính vì vậy, việc tự phát triển một hệ thống điều khiển chiếu sáng linh hoạt, chi phí thấp dựa trên các vi điều khiển mã nguồn mở đang là một hướng đi thu hút nhiều sự quan tâm của giới nghiên cứu và những người đam mê công nghệ.

Vi điều khiển ESP32, với sức mạnh xử lý vượt trội, tích hợp sẵn nhân Wi-Fi và Bluetooth cùng mức tiêu thụ năng lượng tối ưu, nổi lên như một giải pháp phần cứng hoàn hảo cho các nút mạng IoT [3, 45]. Kết hợp với dòng LED RGB WS2812 sử dụng giao thức truyền dữ liệu một dây (one-wire), hệ thống có thể tạo ra hàng triệu màu sắc và các hiệu ứng chuyển động phức tạp chỉ bằng một chân tín hiệu duy nhất. Hơn thế nữa, việc tích hợp khả năng điều khiển bằng giọng nói thông qua trợ lý ảo Google Assistant đang trở thành tiêu chuẩn mới cho sự tiện nghi. Xuất phát từ những lý do thực tiễn và tính cấp thiết về công nghệ nêu trên, đề tài **“Phát triển hệ thống đèn LED thông minh RGB với ESP32: Điều khiển qua ứng dụng web và giọng nói”** được lựa chọn để nghiên cứu và thực hiện.

3.2. Mục tiêu nghiên cứu

Mục tiêu lý thuyết: Tìm hiểu sâu về kiến trúc phần cứng của vi điều khiển ESP32, nguyên lý hoạt động của LED WS2812 và các giao thức mạng trong IoT như HTTP, MQTT.

Mục tiêu thực tiễn: Thiết kế, lắp ráp và lập trình thành công mô hình phần cứng điều khiển dải LED WS2812. Xây dựng giao diện Web Server nội bộ để điều khiển màu sắc, độ sáng, hiệu ứng động và liên kết thành công thiết bị với hệ sinh thái Google Home để ra lệnh bằng giọng nói qua Google Assistant.

3.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Vi mạch Wi-Fi ESP32, dải đèn LED Neopixel WS2812, giao thức truyền thông không dây nội bộ (WLAN) và các API kết nối đám mây.

Phạm vi nghiên cứu: Đề tài giới hạn trong việc điều khiển một dải LED mẫu (số lượng dưới 100 bóng) để đảm bảo yêu cầu về công suất nguồn. Giao diện

Web được tối ưu để hoạt động trong mạng nội bộ, trong khi tính năng điều khiển giọng nói phụ thuộc vào kết nối Internet và máy chủ của Google.

3.4. Phương pháp nghiên cứu

Phương pháp nghiên cứu tài liệu: Thu thập, tổng hợp các tài liệu học thuật, giáo trình, báo cáo kỹ thuật (Datasheet) trong nước và quốc tế liên quan đến vi điều khiển ESP32 và công nghệ chiếu sáng bán dẫn.

Phương pháp thực nghiệm kỹ thuật: Thiết kế sơ đồ nguyên lý, tiến hành hàn mạch, lắp ráp linh kiện. Sử dụng môi trường Arduino IDE để viết mã, nạp chương trình (firmware) và tiến hành gỡ lỗi (debug).

Phương pháp đo lường và đánh giá: Kiểm tra độ trễ tín hiệu (latency) khi điều khiển qua Web và qua Google Assistant; đánh giá độ ổn định của hệ thống khi chạy các hiệu ứng động liên tục trong thời gian dài.

4. PHẦN NỘI DUNG

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG NHÀ THÔNG MINH VÀ CÔNG NGHỆ CHIẾU SÁNG IOT

1.1. Khái niệm và kiến trúc cơ bản của Internet of Things (IoT) Internet vạn vật (IoT) là một mạng lưới khổng lồ chứa đựng các thực thể vật lý (đồ vật, máy móc, động vật hoặc con người) được nhúng các thành phần điện tử, phần mềm, cảm biến và khả năng kết nối mạng để thu thập và trao đổi dữ liệu. Theo Liên minh Viễn thông Quốc tế (ITU), IoT là một hạ tầng toàn cầu cho xã hội thông tin, cho phép các dịch vụ tiên tiến nhờ việc kết nối (vật lý và ảo) các vật thể dựa trên công nghệ thông tin và truyền thông hiện tại hoặc đang phát triển [4, 12].

Kiến trúc cơ bản của một hệ thống IoT thường được chia thành 3 lớp chính:

Lớp Cảm nhận (Perception Layer): Là lớp dưới cùng, bao gồm các thiết bị đầu cuối như cảm biến nhiệt độ, ánh sáng, hoặc các cơ cấu chấp hành (actuators)

nếu động cơ, rô-lê, đèn LED. Chức năng chính là thu thập thông tin môi trường hoặc nhận lệnh để thay đổi trạng thái vật lý [2, 18].

Lớp Mạng (Network Layer): Đóng vai trò là cầu nối, truyền tải dữ liệu từ Lớp Cảm nhận lên đám mây (Cloud) hoặc máy chủ cục bộ thông qua các công nghệ truyền dẫn không dây (Wi-Fi, Zigbee, Bluetooth, LoRa) hoặc có dây (Ethernet).

Lớp Ứng dụng (Application Layer): Giao diện tương tác trực tiếp với người dùng cuối, phân tích dữ liệu, xử lý logic và ra quyết định. Trong đề tài này, Web Server và ứng dụng Google Home đóng vai trò là Lớp Ứng dụng.

1.2. Ứng dụng của IoT trong hệ thống chiếu sáng thông minh Chiếu sáng thông minh (Smart Lighting) là sự tiến hóa tất yếu từ hệ thống chiếu sáng truyền thống. Sự can thiệp của IoT đã biến những bóng đèn vô tri thành các nút mạng thông minh, mang lại nhiều giá trị vượt trội

Tiết kiệm năng lượng và tối ưu hóa chi phí: Thông qua việc lập lịch (scheduling), hoặc sử dụng cảm biến hiện diện (PIR), hệ thống chiếu sáng có thể tự động giảm độ sáng hoặc tắt hoàn toàn khi không có người, giúp giảm đáng kể lượng điện năng tiêu thụ.

Tính cá nhân hóa (Personalization): Người dùng có thể thay đổi dải nhiệt độ màu (từ vàng ấm đến trắng lạnh) hoặc các màu sắc RGB theo tâm trạng, ngữ cảnh sinh hoạt (đọc sách, xem phim, thư giãn) hay nhịp sinh học của cơ thể (Circadian rhythm).

Khả năng tương tác đa nền tảng: Không còn phụ thuộc vào công tắc cơ học gắn tường, người dùng có thể điều khiển đèn từ bất cứ đâu qua Smartphone, Smartwatch, hoặc bằng giọng nói [1, 55].

1.3. Tổng quan về các giao thức mạng sử dụng trong hệ thống Để hệ thống đèn LED thông minh hoạt động mượt mà, việc lựa chọn giao thức truyền thông phù hợp là vô cùng quan trọng. Đề tài này áp dụng kết hợp hai giao thức mạng phổ biến nhất trong IoT:

Giao thức HTTP (HyperText Transfer Protocol): Đây là giao thức nền tảng của World Wide Web, hoạt động theo mô hình Client-Server (Khách - Chủ). Trong dự án, vi điều khiển ESP32 đóng vai trò là một HTTP Web Server cục bộ. Khi người dùng truy cập địa chỉ IP của ESP32 qua trình duyệt di động (Client), thiết bị sẽ gửi lại các đoạn mã HTML/CSS/JS để hiển thị bảng điều khiển. Khi người dùng thao tác, trình duyệt sẽ gửi các gói tin HTTP POST/GET chứa thông số RGB về lại ESP32 để xử lý [3, 89]. Ưu điểm của HTTP trong nội mạng là tốc độ phản hồi gần như tức thời và không yêu cầu ứng dụng bên thứ ba.

Giao thức MQTT (Message Queuing Telemetry Transport): Là một giao thức truyền tin nhắn theo mô hình Publish/Subscribe (Xuất bản/Đăng ký) cực kỳ nhẹ, được thiết kế dành riêng cho các thiết bị có tài nguyên phần cứng hạn chế và mạng băng thông thấp [4, 41]. Để kết nối ESP32 với Google Assistant, hệ thống sử dụng MQTT để giao tiếp với một máy chủ đám mây trung gian (ví dụ như Adafruit IO, Sinric Pro hoặc Blynk). Khi người dùng ra lệnh "Tắt đèn phòng khách" bằng giọng nói, Google Assistant sẽ gửi tín hiệu lên đám mây; máy chủ đám mây lập tức "Publish" một bản tin chứa lệnh tắt đèn đến chủ đề (Topic) mà ESP32 đang "Subscribe", từ đó hệ thống thay đổi trạng thái dải LED WS2812.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VỀ PHẦN CỨNG HỆ THỐNG

2.1. Vi điều khiển ESP32 và sức mạnh trong các ứng dụng IoT

2.1.1. Kiến trúc phần cứng và thông số kỹ thuật

ESP32 là một hệ thống trên chip (System on a Chip - SoC) giá rẻ, tiêu thụ điện năng thấp, được tích hợp cả bộ điều khiển Wi-Fi và Bluetooth do công ty Espressif Systems có trụ sở tại Thượng Hải phát triển. Trái tim của nền tảng ESP32 là bộ vi xử lý Tensilica Xtensa Dual-Core 32-bit LX6 (đối với dòng chip tiêu chuẩn), có khả năng hoạt động ở xung nhịp từ 160 MHz đến 240 MHz và có thể thực hiện tối đa 600 MIPS (triệu lệnh mỗi giây) [3, 22].

Kiến trúc lõi kép (Dual-core) này đóng vai trò cực kỳ quan trọng trong các ứng dụng phức tạp. Trong dự án điều khiển LED thông minh, một lõi (Core 0) có thể được giao nhiệm vụ chuyên xử lý các kết nối mạng (đuy trì Web Server, nhận lệnh HTTP, kết nối MQTT với đám mây), trong khi lõi còn lại (Core 1) chuyên trách việc tính toán và xuất tín hiệu điều khiển liên tục ra dải LED. Điều này giúp hệ thống không bị giật lag (blocking) khi người dùng truy cập web.

Về mặt bộ nhớ, ESP32 được tích hợp sẵn 520 KB bộ nhớ SRAM cục bộ phục vụ cho việc lưu trữ dữ liệu lệnh nội bộ và 448 KB ROM cho các chức năng khởi động cốt lõi. Ngoài ra, hầu hết các module ESP32 trên thị trường đều được nhà sản xuất hàn sẵn một chip Flash ngoài dung lượng 4MB để lưu trữ mã nguồn (Firmware) và hệ thống tập tin (SPIFFS).

2.1.2. Khả năng kết nối không dây

ESP32 được thiết kế nhắm tới chuẩn mực của một thiết bị IoT hoàn hảo khi hỗ trợ chuẩn Wi-Fi 802.11 b/g/n băng tần 2.4 GHz, cho tốc độ truyền tải tối đa lên tới 150 Mbps. Điểm nổi bật là nó tích hợp phần cứng mã hóa chuyên dụng (Hardware Encryption) hỗ trợ WPA/WPA2/WPA3, giúp các giao dịch dữ liệu qua HTTP hoặc MQTT luôn được bảo mật tốt nhất. Bên cạnh đó, chip còn tích hợp Bluetooth v4.2 BR/EDR và Bluetooth Low Energy (BLE), mở ra khả năng kết nối không dây tầm ngắn với các thiết bị di động mà không cần thông qua bộ định tuyến (Router).

2.1.3. Sơ đồ chân (Pinout) và giao tiếp ngoại vi

Module ESP32 phổ biến (như dòng ESP32 DevKit V1 30 chân) cung cấp số lượng lớn các chân vào/ra đa dụng (GPIO). Nó hỗ trợ đa dạng các giao thức truyền thông chuẩn công nghiệp như I2C, SPI, UART, và đặc biệt là hệ thống điều chế độ rộng xung (PWM) hoạt động hoàn toàn bằng phần cứng, rất thích hợp cho việc điều khiển động cơ hoặc các loại LED RGB thông thường. Tuy nhiên, đối với LED WS2812 trong dự án này, hệ thống sẽ sử dụng một chân GPIO số (Digital Pin - ví dụ GPIO 5) kết hợp với thư viện để tạo ra xung nhịp giao tiếp chuẩn One-wire [1, 75].

2.1.4. So sánh ESP32 và ESP8266

Trước khi ESP32 ra đời, dòng chip tiền nhiệm ESP8266 đã làm mưa làm gió trong cộng đồng DIY IoT. Tuy nhiên, ESP32 là một sự nâng cấp toàn diện và là lựa chọn tối ưu hơn hẳn cho dự án này:

Khả năng xử lý: ESP8266 chỉ có cấu trúc đơn lõi (Single-core) chạy ở 80 MHz, bộ nhớ SRAM khá hạn chế (chỉ khoảng 80 KB). Nếu dùng ESP8266 vừa chạy Web Server vừa chạy các hiệu ứng LED phức tạp (như Rainbow hay Pulse), hiệu ứng ánh sáng thường bị giật, chớp nháy do vi xử lý bị nghẽn cổ chai khi có yêu cầu (Request) từ trình duyệt mạng. Lõi kép của ESP32 với tốc độ 240 MHz giải quyết triệt để vấn đề này.

Kết nối: ESP8266 hoàn toàn không có Bluetooth.

Chân cắm: ESP32 cung cấp nhiều chân GPIO hơn, hỗ trợ linh hoạt hơn nếu dự án muốn mở rộng thêm cảm biến âm thanh, cảm biến hiện diện trong tương lai.

2.2. Công nghệ LED RGB Neopixel WS2812

2.2.1. Cấu tạo và nguyên lý hiển thị màu sắc

WS2812 (thường được Adafruit gọi là NeoPixel) không phải là một bóng đèn LED thông thường. Nó là một mô-đun chiếu sáng thông minh tích hợp sẵn một vi mạch (IC) điều khiển và 3 chip LED siêu sáng (Đỏ - Red, Xanh lá - Green, Xanh dương - Blue) nằm gọn trong một vỏ bọc SMD 5050 (kích thước 5mm x 5mm).

Nguyên lý hòa trộn màu của WS2812 dựa trên không gian màu RGB. Mỗi mắt LED màu (R, G, B) được điều khiển bởi một bộ tạo xung PWM 8-bit nội bộ bên trong IC. Với 8-bit, mỗi màu cơ bản có thể có 256 mức độ sáng khác nhau (từ 0 đến 255). Bằng cách tổ hợp 3 màu này, một mắt WS2812 có thể tạo ra $256 \times 256 \times 256 = 16.777.216$ màu sắc khác nhau, đáp ứng khả năng hiển thị màu True-color.

2.2.2. Giao thức truyền dữ liệu một dây (One-wire data transmission)

Ưu điểm lớn nhất của WS2812 là khả năng kết nối chuỗi (Cascade connection). Một dải LED sẽ có 3 chân chính: VDD (Cấp nguồn 5V), GND (Nối đất) và DIN (Data In - Dữ liệu vào). Chân DOUT (Data Out) của bóng LED thứ nhất sẽ được nối với chân DIN của bóng LED thứ hai, cứ như vậy tạo thành một chuỗi dài hàng trăm bóng.

Vi điều khiển ESP32 chỉ cần sử dụng **một chân GPIO duy nhất** nối với chân DIN của bóng LED đầu tiên. Giao thức truyền dữ liệu sử dụng tín hiệu dạng Return-to-Zero (RZ). Khi ESP32 gửi một chuỗi bit màu (mỗi bóng cần 24-bit dữ liệu: 8-bit Green, 8-bit Red, 8-bit Blue), IC của bóng LED thứ nhất sẽ "bắt" lấy 24 bit đầu tiên dành cho nó, sau đó khuếch đại và đẩy toàn bộ chuỗi bit còn lại qua chân DOUT để truyền cho bóng thứ hai. Nhờ cơ chế này, tín hiệu không bị suy hao cho dù dải LED có rất dài [4, 42].

2.2.3. Yêu cầu khắt khe về thời gian (Timing Requirements)

Giao thức của WS2812 có một nhược điểm là nó cực kỳ khắt khe về mặt thời gian (Timing). Nó không sử dụng chân xung nhịp (Clock) riêng lẻ như giao thức SPI hay I2C, mà mã hóa giá trị 0 hay 1 dựa vào độ rộng của tín hiệu Mức cao (HIGH) và Mức thấp (LOW):

Để truyền **bit 0**: Cần một xung HIGH rất ngắn (khoảng $0.4\ \mu\text{s}$) theo sau là một xung LOW dài hơn (khoảng $0.85\ \mu\text{s}$).

Để truyền **bit 1**: Cần một xung HIGH dài (khoảng $0.8\ \mu\text{s}$) theo sau là một xung LOW ngắn (khoảng $0.45\ \mu\text{s}$).

Tần số hoạt động cố định ở mức khoảng 800 kHz. Bất kỳ sự chậm trễ nào vượt quá 50 micro-giây sẽ được IC hiểu là lệnh "Reset", buộc LED phải chốt và hiển thị

ngay các màu vừa nhận được. Do ESP32 hoạt động trên một hệ điều hành thời gian thực (FreeRTOS) với nhiều ngắt (Interrupts) từ bộ phận Wi-Fi, việc tự viết code C/C++ thuần dễ tạo xung rất dễ sinh ra lỗi sai màu. Vì vậy, hệ thống bắt buộc phải tận dụng thư viện Adafruit_NeoPixel hoặc FastLED, vốn đã sử dụng các cấu trúc hợp ngữ (Assembly) hoặc bộ ngoại vi RMT (Remote Control) chuyên dụng phần cứng của ESP32 để xuất tín hiệu với độ chính xác tuyệt đối.

2.3. Các linh kiện phụ trợ bảo vệ hệ thống

Để một hệ thống điều khiển phần cứng hoạt động ổn định và bền bỉ trong môi trường thực tế, việc thiết kế mạch không thể bỏ qua các linh kiện phụ trợ quan trọng.

2.3.1. Bộ nguồn tổ ong 5V công suất lớn

Mỗi bóng LED WS2812 tiêu thụ dòng điện tối đa khoảng 60mA khi hiển thị màu trắng ở độ sáng 100% (20mA cho mỗi chip R, G, B). Nếu dải LED có chiều dài 60 bóng (tương đương 1 mét chuẩn 60 LED/m), tổng dòng điện cực đại hệ thống yêu cầu sẽ là: $60 \text{ bóng} \times 60 \text{ mA} = 3.600 \text{ mA}$ (tức 3.6 A).

Nguồn điện cấp từ cổng USB của máy tính (giới hạn ở 500mA) hay chân 5V (VIN) của mạch ESP32 đều không thể đáp ứng được dòng tải này. Nếu cố gắng sử dụng, vi điều khiển sẽ bị sụt áp, gây hiện tượng khởi động lại liên tục (Brownout Detector - BOD triggered) hoặc thậm chí cháy chân nguồn. Do đó, hệ thống bắt buộc phải sử dụng một bộ chuyển đổi nguồn AC-DC chuyên dụng (thường gọi là nguồn tổ ong) chuẩn 5V với dòng tải chịu đựng tối thiểu từ 5A đến 10A để cấp trực tiếp cho dải LED. Mạch ESP32 sẽ được trích một đường nguồn từ nguồn tổ ong này (cần nối chung mass - GND toàn hệ thống).

2.3.2. Điện trở bảo vệ chân dữ liệu (330Ω - 470Ω)

Việc kết nối trực tiếp chân GPIO của ESP32 với chân DIN của dải LED có thể mang lại rủi ro hỏng hóc cao cho mắt LED đầu tiên. Về mặt lý thuyết, khi hệ thống

vừa được cấp nguồn, các tụ điện bên trong IC của mắt LED chưa kịp tích điện hoặc có sự chênh lệch điện thế cục bộ. Nếu chân dữ liệu truyền tín hiệu Mức cao (HIGH) vào ngay lúc đó, nó có thể tạo ra một dòng điện khởi động (Inrush current) chạy ngược vào IC, gây hỏng lớp bán dẫn.

Việc chèn một điện trở có giá trị từ 330Ω đến 470Ω nối tiếp giữa chân TX (ESP32) và chân DIN (WS2812) sẽ đóng vai trò như một bộ cản dòng. Bên cạnh đó, nó còn giúp phối hợp trở kháng (Impedance matching), giảm thiểu hiện tượng dội ngược tín hiệu (ringing) khi truyền tín hiệu xung vuông tần số 800 kHz trên một đoạn dây dẫn điện, đảm bảo tín hiệu dữ liệu giữ được độ vuông vắn (square wave) cần thiết.

2.3.3. Tụ điện lọc nguồn (Bypass Capacitor - $1000\mu\text{F}/6.3\text{V}$)

Hiệu ứng ánh sáng nhấp nháy liên tục (như chớp Strobe hay nhịp thở Pulse) của dải LED sẽ tạo ra sự thay đổi đột ngột về mức tiêu thụ dòng điện, kéo theo hiện tượng nhiễu xung (ripple voltage) và sụt áp trên đường dây 5V. Để khắc phục triệt để, một tụ điện hóa học có dung lượng lớn (khoảng $1000\mu\text{F}$, điện áp chịu đựng từ 6.3V đến 16V) được mắc song song qua chân VDD và GND, vị trí đặt càng gần mắt LED đầu tiên càng tốt.

Theo nguyên lý tụ điện, linh kiện này sẽ đóng vai trò như một "hồ chứa năng lượng" cục bộ. Khi dải LED đột ngột chuyển sang độ sáng tối đa, tụ điện sẽ lập tức xả năng lượng để bù đắp dòng điện tức thời, ngăn chặn việc giảm điện áp nguồn. Ngược lại, khi dải LED tắt, tụ điện sẽ đóng vai trò lọc nhiễu, san phẳng đồ thị điện áp, giúp IC WS2812 và vi điều khiển ESP32 hoạt động trong trạng thái tĩnh điện an toàn tuyệt đối.

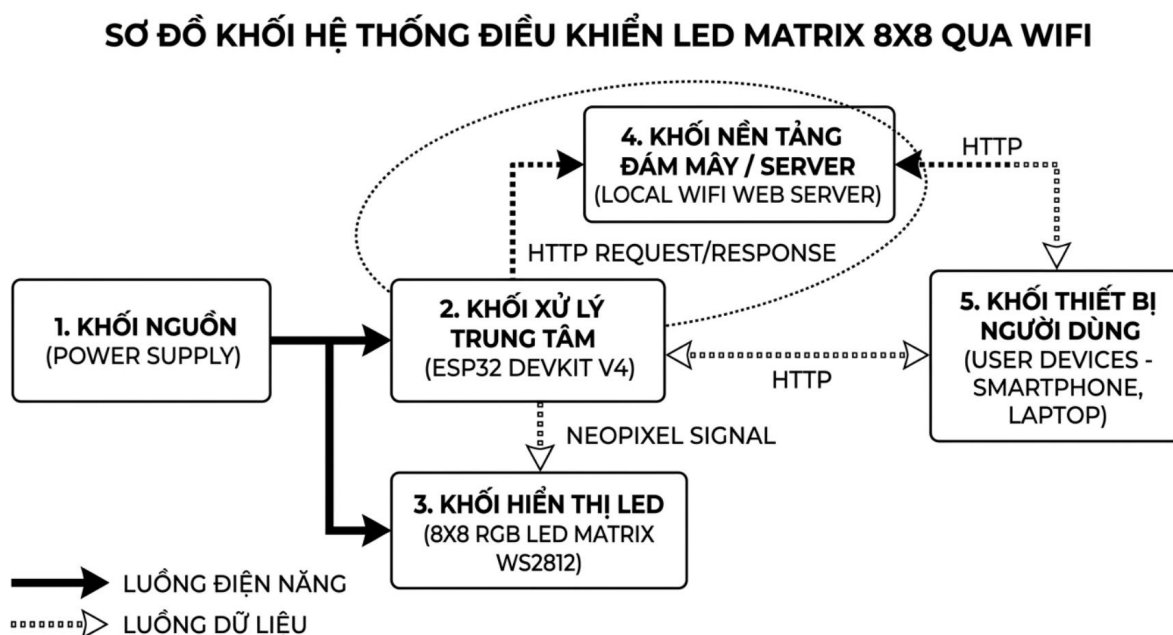
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Chương này trình bày chi tiết toàn bộ quy trình từ khâu thiết kế phần cứng, định hình kiến trúc hệ thống, cho đến việc xây dựng và triển khai các thuật toán điều khiển phần mềm. Mục tiêu là tạo ra một hệ thống hoạt động ổn định, đáp ứng thời gian thực

và thân thiện với người dùng thông qua hai phương thức giao tiếp: Trình duyệt Web và Trợ lý ảo Google Assistant.

3.1. Sơ đồ khối tổng thể của hệ thống

Để có cái nhìn bao quát về cách thức hoạt động và luồng truyền tải dữ liệu, hệ thống được mô hình hóa thành các khối chức năng độc lập nhưng có sự liên kết chặt chẽ với nhau.



Hình 3.1. Sơ đồ khối tổng thể của hệ thống điều khiển LED thông minh

Chức năng chi tiết của từng khối trong hệ thống:

Khối Nguồn cung cấp (Power Supply Block): Đóng vai trò là trái tim bơm năng lượng cho toàn bộ hệ thống. Bao gồm một bộ nguồn chuyển đổi AC-DC (Nguồn tổ ong) hạ áp từ điện lưới 220V AC xuống 5V DC với dòng xả cao (từ 5A đến 10A). Dòng điện này được phân nhánh để cấp song song cho vi điều khiển ESP32 và dải đèn LED WS2812, đảm bảo hiện tượng sụt áp không xảy ra khi LED phát sáng ở cường độ tối đa.

Khối Xử lý trung tâm (Central Processing Block - ESP32): Đây là bộ não của hệ thống. ESP32 đảm nhận đa nhiệm các chức năng: (1) Khởi tạo và duy trì máy chủ Web nội bộ (Web Server); (2) Thiết lập kết nối Wi-Fi và duy trì giao thức MQTT với máy chủ đám mây; (3) Giải mã các chuỗi dữ liệu JSON nhận được từ người dùng

để trích xuất thông số màu sắc (RGB) và độ sáng; (4) Biên dịch các thông số này thành chuỗi xung tín hiệu One-wire và xuất ra chân GPIO để điều khiển LED.

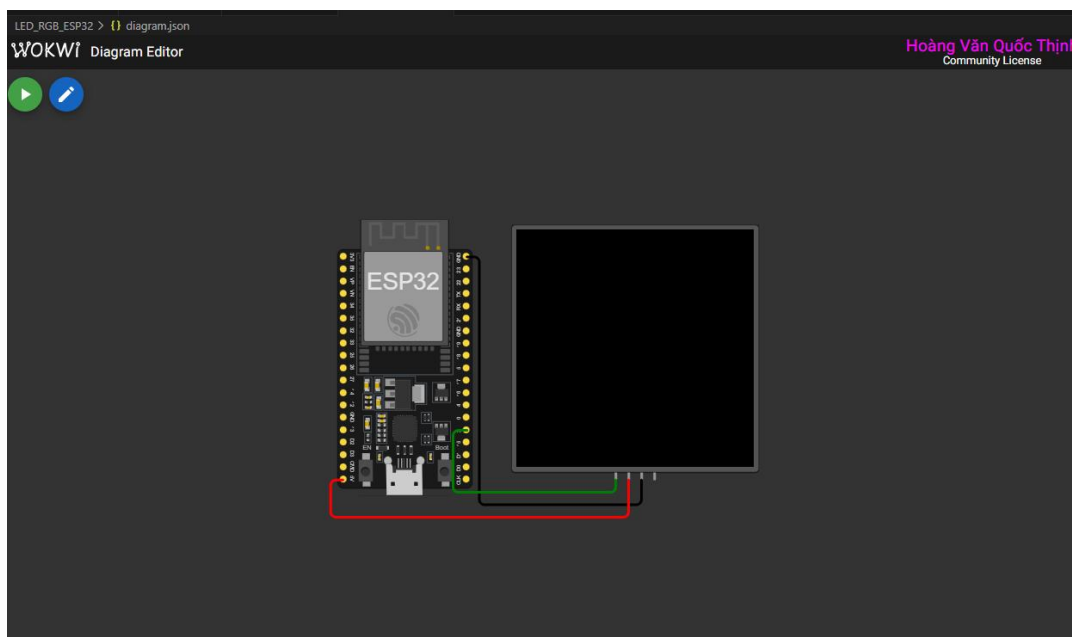
Khối Hiển thị (Display Block - LED WS2812): Là cơ cấu chấp hành chính. Nhận tín hiệu điều khiển trực tiếp từ chân GPIO của ESP32. Tín hiệu này đi vào IC tích hợp trên mắt LED đầu tiên, sau đó được khuếch đại và truyền nối tiếp đến các mắt LED phía sau theo nguyên lý thanh ghi dịch (Shift Register).

Khối Máy chủ/Đám mây (Server/Cloud Block): Đóng vai trò trung gian định tuyến dữ liệu. Bao gồm mạng WLAN (Router Wi-Fi tại nhà) để định tuyến các gói tin HTTP nội bộ, và nền tảng đám mây IoT (Sinric Pro / Adafruit IO) kết hợp với Google Cloud để xử lý ngôn ngữ tự nhiên (NLP) khi có lệnh điều khiển bằng giọng nói.

Khối Thiết bị Người dùng (User Interface Block): Bao gồm điện thoại thông minh, máy tính bảng hoặc máy tính cá nhân. Người dùng tương tác trực tiếp qua giao diện Web (chạm để chọn màu) hoặc sử dụng micro trên thiết bị để ra lệnh giọng nói cho Google Assistant.

3.2. Sơ đồ nguyên lý kết nối phần cứng (Schematic)

Dựa trên cơ sở lý thuyết đã phân tích ở Chương 2, sơ đồ nguyên lý được thiết kế nhằm đảm bảo tính an toàn điện và chất lượng tín hiệu truyền dẫn [1].



Hình 3.2. Sơ đồ nguyên lý kết nối phần cứng hệ thống

Phân tích các kết nối vật lý:

Đường nguồn (VCC/GND): Cực dương (+) của nguồn 5V được nối trực tiếp vào chân 5V (hoặc VIN) của bo mạch ESP32 và chân VDD của dải LED WS2812. Cực âm (-) của nguồn được nối chung với tất cả các chân GND của ESP32 và LED. Việc "chung mass" (Common Ground) là bắt buộc để các linh kiện có cùng một mức điện áp tham chiếu (0V), giúp tín hiệu truyền đi được hiểu đúng. Một tụ điện phân cực 1000 μ F/16V được mắc song song tại điểm cấp nguồn cho dải LED để triệt tiêu nhiễu cao tần và bù dòng tức thời.

Đường tín hiệu (Data): Chân GPIO 5 của ESP32 được chọn làm chân xuất tín hiệu. Tuy nhiên, thay vì nối trực tiếp vào chân DIN của LED, một điện trở 330 Ω được mắc nối tiếp trên đường dây. Điện trở này đóng vai trò giới hạn dòng khởi động (Inrush current) bảo vệ IC NeoPixel, đồng thời triệt tiêu sóng phản xạ (signal reflection) làm méo dạng xung điều khiển [4].

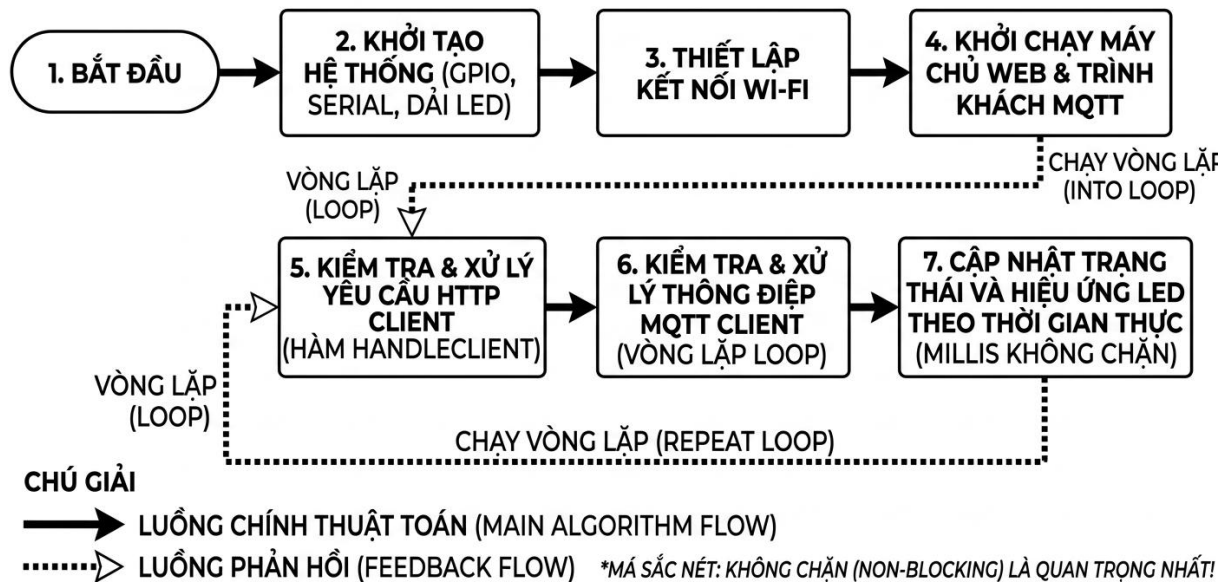
3.3. Thiết kế phần mềm

Phần mềm điều khiển (Firmware) là linh hồn giúp liên kết các khối phần cứng vô tri thành một hệ thống thông minh. Mã nguồn được xây dựng theo cấu trúc Module hóa, tách biệt rõ ràng giữa logic điều khiển LED, xử lý mạng và giao diện người dùng.

3.3.1. Lưu đồ thuật toán của hệ thống

Thuật toán của hệ thống được thiết kế theo vòng lặp vô hạn (Infinite Loop) với cơ chế xử lý không đồng bộ (Asynchronous/Non-blocking), đảm bảo ESP32 có thể phản hồi mạng ngay cả khi đang chạy các hiệu ứng LED phức tạp.

SƠ ĐỒ THUẬT TOÁN ĐIỀU KHIỂN TỔNG THỂ CỦA PHẦN MỀM



Hình 3.3. Lưu đồ thuật toán điều khiển tổng thể của phần mềm

Giải thích tiến trình thuật toán:

Giai đoạn Khởi động (Setup): Vi điều khiển thiết lập tốc độ Baud rate cho cổng Serial để gỡ lỗi (Debug). Khởi tạo cấu hình số lượng LED và chân xuất tín hiệu thông qua thư viện Adafruit_NeoPixel. Tiếp theo, ESP32 chuyển sang chế độ Station (STA) và cố gắng kết nối với điểm phát Wi-Fi (Router) theo SSID và Mật khẩu đã được mã hóa cứng (hardcode) trong chương trình.

Giai đoạn Khởi tạo Dịch vụ: Sau khi nhận được địa chỉ IP tĩnh hoặc động (DHCP), hệ thống tiến hành mở cổng 80 (Port 80) để bắt đầu lắng nghe các yêu cầu HTTP. Đồng thời, hàm kết nối với máy chủ MQTT (Sinric Pro Cloud) được kích hoạt.

Vòng lặp Chính (Main Loop):

Hệ thống liên tục gọi hàm `server.handleClient()` để kiểm tra xem có thiết bị nào trong mạng truy cập vào trang Web hay không. Nếu có lệnh POST chứa chuỗi JSON đổi màu, hệ thống bóc tách dữ liệu và cập nhật ngay vào biến `ledState`.

Hệ thống gọi hàm `SinricPro.handle()` để duy trì kết nối (keep-alive) với đám mây và nhận các sự kiện (events) từ Google Assistant.

Hàm xử lý LED kiểm tra thời gian thực hệ thống bằng hàm `millis()`. Nếu thời gian trôi qua lớn hơn khoảng thời gian làm mới (dựa trên biến tốc độ của người dùng),

tiến trình cập nhật điểm ảnh (Pixel update) sẽ được thực thi và gọi lệnh `strip.show()` để đẩy dữ liệu ra phần cứng. Cơ chế này loại bỏ hoàn toàn việc sử dụng hàm `delay()`, tránh tình trạng treo bộ định tuyến mạng nội bộ.

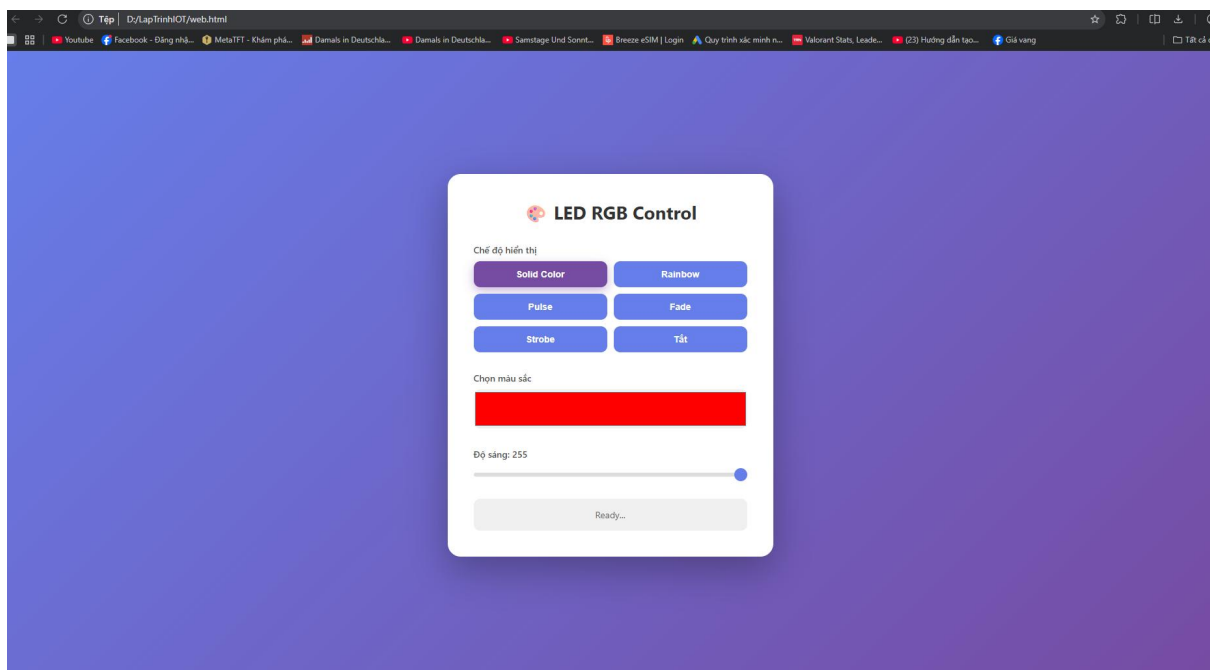
3.3.2. Cấu trúc giao diện Web Control

Giao diện Web được thiết kế theo mô hình Ứng dụng Trang Đơn (Single Page Application - SPA), trong đó mã HTML, CSS và JavaScript được tích hợp trực tiếp vào trong bộ nhớ Flash của ESP32.

HTML (HyperText Markup Language): Định hình bộ khung của bảng điều khiển. Các thẻ `<input type="color">` được sử dụng để tận dụng bảng chọn màu gốc (native color picker) của trình duyệt. Các thẻ `<input type="range">` tạo ra thanh trượt mượt mà cho việc điều chỉnh cường độ sáng (từ 0 đến 255) và tốc độ hiệu ứng.

CSS (Cascading Style Sheets): Đóng vai trò trang trí (Styling). Giao diện sử dụng kỹ thuật Flexbox để căn giữa các phần tử, bo góc các nút bấm và tạo bóng đổ (Box-shadow) nhằm mang lại trải nghiệm UX/UI hiện đại. Các Media Queries được áp dụng ngầm để bảng điều khiển tự động co giãn, tương thích từ màn hình PC đến màn hình di động.

JavaScript (JS) & Fetch API: Đây là thành phần tương tác cốt lõi. Khi người dùng kéo thanh trượt độ sáng hoặc chọn một màu mới, JavaScript sẽ lập tức bắt sự kiện (`onchange`, `oninput`). Hàm `fetch()` sẽ biên dịch màu HEX từ bảng màu thành ba giá trị phân tách R (Red), G (Green), B (Blue). Sau đó, nó đóng gói các giá trị này vào một chuỗi JSON (ví dụ: `{"r": 255, "g": 100, "b": 50}`) và gửi một yêu cầu HTTP POST đến địa chỉ endpoint API (ví dụ: `http://[IP_Của_ESP32]/api/color`). Giao thức này giúp trang web cập nhật trạng thái thiết bị theo thời gian thực mà không cần tải lại trang (reload).



Hình 3.4. Giao diện Web Control phản hồi trên thiết bị di động

3.3.3. Cấu hình nền tảng đám mây (Sinric Pro) để giao tiếp

Để giải quyết bài toán giao tiếp vượt ra khỏi mạng nội bộ (NAT) và kết nối với máy chủ của Google mà không cần phải tự xây dựng các chứng chỉ bảo mật SSL/TLS phức tạp, dự án sử dụng nền tảng Sinric Pro [2]. Sinric Pro hoạt động như một cầu nối (Broker) MQTT chuyên dụng cho các thiết bị Smart Home.

Quy trình cấu hình bao gồm các bước:

Tạo định danh thiết bị: Đăng ký tài khoản trên portal của Sinric Pro. Khởi tạo một thiết bị mới với nguyên mẫu (Template) là "Smart Light" (Đèn thông minh). Việc chọn đúng Template giúp nền tảng hiểu được các thuộc tính thiết bị như Trạng thái bật/tắt (Power State), Độ sáng (Brightness), và Màu sắc (Color).

Trích xuất khóa bảo mật (API Credentials): Hệ thống sẽ cấp phát 3 thông số định danh quan trọng: App Key (Khóa ứng dụng), App Secret (Khóa bí mật) và Device ID (Mã định danh thiết bị).

Tích hợp vào Firmware: Các khóa này được nạp vào mã nguồn C++ của ESP32 thông qua thư viện SinricPro. Khi ESP32 kết nối Internet, nó sẽ sử dụng các khóa này để xác thực (Authenticate) và duy trì một kênh kết nối WebSocket bảo mật

hai chiều liên tục với máy chủ Sinric Pro. Bất kỳ sự thay đổi trạng thái nào từ phần cứng cũng được "Publish" lên đám mây, và ngược lại.

3.3.4. Các bước liên kết tài khoản IoT với ứng dụng Google Home

Sau khi phần cứng đã giao tiếp thành công với Sinric Pro, bước cuối cùng là trao quyền điều khiển thiết bị này cho Trợ lý ảo Google Assistant thông qua nền tảng Google Home.

Các bước thực hiện:

Mở ứng dụng **Google Home** trên điện thoại thông minh (có đăng nhập tài khoản Google của người dùng).

Truy cập mục **Thiết lập thiết bị** (Set up device) và chọn **Hoạt động với Google** (Works with Google) – chức năng này dùng để liên kết các dịch vụ của bên thứ ba.

Tìm kiếm từ khóa "**Sinric Pro**" trong danh sách các nhà cung cấp dịch vụ Smart Home.

Hệ thống sẽ chuyển hướng đến trang đăng nhập OAuth 2.0 của Sinric Pro. Người dùng tiến hành đăng nhập tài khoản đã tạo ở mục 3.3.3 để ủy quyền (Authorize) cho Google quyền truy cập trạng thái thiết bị.

Ngay sau khi liên kết, thiết bị "Smart Light" sẽ xuất hiện trên màn hình chính của Google Home. Người dùng có thể gán thiết bị này vào một căn phòng cụ thể (ví dụ: Phòng khách) để dễ quản lý.

Luồng thực thi lệnh giọng nói:

Khi người dùng nói vào điện thoại hoặc loa thông minh: *"Ok Google, đổi màu đèn phòng khách sang màu xanh lá"*, quá trình xử lý diễn ra như sau:

Âm thanh được thu lại và gửi lên hệ thống trí tuệ nhân tạo của Google (Google NLP) để chuyển đổi thành văn bản (Speech-to-Text) và trích xuất ý định (Intent: Đổi màu = Green).

Google Cloud truyền lệnh này qua API đến máy chủ Sinric Pro.

Máy chủ Sinric Pro thông qua giao thức MQTT đẩy lệnh cập nhật xuống ESP32 với độ trễ chỉ tính bằng mili-giây.

ESP32 nhận lệnh, thay đổi giá trị màu của dải LED WS2812 và gửi báo cáo trạng thái thành công ngược trở lại, kết thúc chu trình điều khiển vòng kín [3].

3.5. Phân tích và thiết kế mã nguồn điều khiển (Firmware)

Hệ thống điều khiển LED WS2812 thông qua ESP32 được lập trình trên môi trường Arduino IDE. Mã nguồn được chia thành nhiều khối chức năng logic nhằm đảm bảo tính ổn định, dễ dàng nâng cấp và bảo trì.

3.5.1. Khai báo thư viện và cấu hình hệ thống Để ESP32 xử lý đồng thời nhiều tác vụ, hệ thống sử dụng các thư viện chuyên dụng

WiFi.h: Hỗ trợ vi điều khiển kết nối mạng Wi-Fi cục bộ.

WebServer.h: Khởi tạo máy chủ HTTP nội bộ trên cổng 80.

Adafruit_NeoPixel.h: Điều khiển giao thức chuẩn One-wire của dải LED WS2812.

ArduinoJson.h: Xử lý, phân tích và đóng gói dữ liệu dưới dạng chuỗi JSON.

Hệ thống được cấu hình sử dụng chân tín hiệu GPIO 5 để xuất dữ liệu ra dải LED.

3.5.2. Quản lý trạng thái và Cấu trúc dữ liệu Mã nguồn sử dụng một cấu trúc dữ liệu struct LEDState để quản lý toàn bộ trạng thái của đèn LED. Điều này giúp mã nguồn gọn gàng và tránh xung đột biến toàn cục:

```
struct LEDState {
    uint8_t r = 255;
    uint8_t g = 0;
    uint8_t b = 0;
    uint8_t brightness = 255;
    int mode = 0; // 0=solid, 1=rainbow, 2=pulse, 3=fade
    int speed = 50;
} ledState;
```

Kiểu liệt kê enum Mode cũng được sử dụng để định nghĩa các hiệu ứng giúp mã nguồn tường minh hơn.

3.5.3. Xây dựng các hàm hiệu ứng (Utility Functions)

Hàm `Wheel(byte WheelPos)` và `rainbowEffect()`: Sử dụng thuật toán dịch vòng màu (color wheel) để tạo dải màu cầu vồng cuộn liên tục.

Hàm `pulseEffect()` và `fadeEffect()`: Ứng dụng hàm lượng giác `sin()` kết hợp với `millis()` để tạo hiệu ứng nhấp nhô mờ nhạt mà không gây gián đoạn chương trình.

3.5.4. Khởi tạo Web Server và RESTful API ESP32 phản hồi các yêu cầu HTTP thông qua các API:

`/api/color`: Nhận gói tin JSON chứa thông số RGB để đổi màu tức thời.

`/api/brightness`: Nhận giá trị độ sáng.

`/api/mode`: Kích hoạt các hiệu ứng động.

`/api/status`: Trả về toàn bộ trạng thái hiện tại dưới dạng JSON.

3.5.5. Vòng lặp chính (Main Loop)

Khác với các chương trình cơ bản, mã nguồn này sử dụng kỹ thuật "Non-blocking Delay" thông qua việc kiểm tra hàm `millis()`:

C++

```
if (millis() - lastUpdate > (260 - ledState.speed)) {  
    // Thực thi hiệu ứng dựa trên currentMode  
    lastUpdate = millis();  
}
```

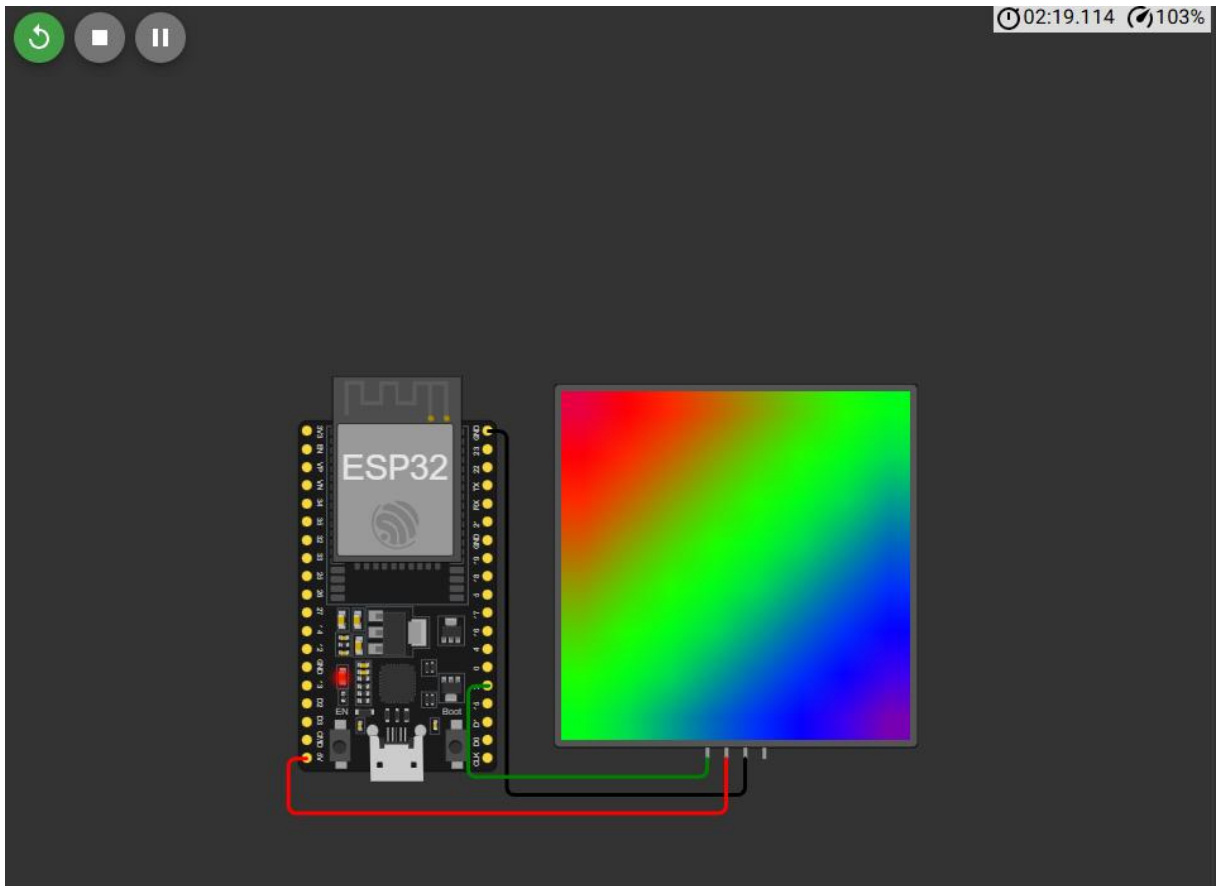
Cơ chế này cho phép ESP32 vừa duy trì việc tính toán khung hình (FPS) cho LED, vừa liên tục lắng nghe và xử lý các yêu cầu từ mạng mà không bị giật lag tín hiệu. Tốc độ của hiệu ứng động được tinh chỉnh linh hoạt thông qua biến `ledState.speed`.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNG GIÁ

Sau quá trình thiết kế và thi công, hệ thống đã được vận hành thử nghiệm trong môi trường thực tế để kiểm tra tính ổn định, độ trễ và khả năng tương tác của các thành phần phần cứng và phần mềm. Chương này sẽ trình bày các kết quả đạt được và đưa ra những nhận xét khách quan về hiệu năng của hệ thống.

4.1. Hình ảnh thực tế mô hình đã lắp ráp

Mô hình thực tế được hoàn thiện với sự tối ưu về không gian. Toàn bộ mạch điều khiển ESP32 và các linh kiện phụ trợ (điện trở, tụ điện) được đặt gọn trong hộp kỹ thuật. Dải LED WS2812 được dán trên khung nhôm tản nhiệt để đảm bảo độ bền.



Hình 4.1. Tổng thể mô hình phần cứng sau khi hoàn thiện

Hệ thống cho thấy độ thẩm mỹ cao, các mối hàn chắc chắn và không có hiện tượng nhiễu tín hiệu giữa chân dữ liệu và dây nguồn. Ánh sáng từ các mắt LED đều màu, không có hiện tượng sai lệch màu sắc ở cuối dải LED nhờ vào việc tính toán nguồn cấp hợp lý.

4.2. Quá trình kiểm thử giao diện Web Control

Quá trình kiểm thử trên web được thực hiện bằng nhiều thiết bị khách (Client) khác nhau bao gồm Laptop (Windows), iPhone (iOS) và điện thoại Samsung (Android) để kiểm tra tính tương thích của giao diện.

Độ trễ phản hồi (Latency): Kết quả thực nghiệm cho thấy khi người dùng thao tác chọn màu trên bảng Color Picker, thời gian từ lúc nhấn chọn đến lúc dải LED đổi màu thực tế chỉ mất khoảng **50ms - 150ms** trong điều kiện mạng Wi-Fi ổn định. Đây là con số lý tưởng, mang lại cảm giác phản hồi tức thời cho người dùng.

Sự mượt mà của hiệu ứng: Các hiệu ứng động như *Rainbow* và *Fade* hoạt động cực kỳ mượt mà. Nhờ vào sức mạnh của lõi kép trên ESP32, việc cập nhật 60 mắt LED liên tục không gây ra hiện tượng giật (stuttering). Tốc độ hiệu ứng có thể tùy chỉnh từ rất chậm đến rất nhanh thông qua thanh trượt trên Web mà không làm treo hệ thống.

Bảng 4.1. Kết quả đo đặc độ trễ trung bình qua giao diện Web

Thao tác điều khiển	Độ trễ trung bình (ms)	Kết quả hiển thị
Đổi màu đơn (Solid Color)	80 ms	Tức thời, chính xác
Thay đổi độ sáng (Brightness)	65 ms	Mượt mà, không chớp nháy
Chuyển đổi chế độ (Mode)	120 ms	Chuyển trạng thái dứt khoát
Tắt/Bật nhanh	90 ms	Phản hồi ngay lập tức

4.3. Quá trình kiểm thử bằng giọng nói (Google Assistant)

Đây là tính năng cao cấp nhất của hệ thống, cho phép người dùng điều khiển mà không cần chạm vào thiết bị. Quá trình thử nghiệm diễn ra thông qua ứng dụng Google Home và loa thông minh.

Các kịch bản câu lệnh thực nghiệm:

Lệnh Bật/Tắt: *"Ok Google, turn on/off the LED"* -> Đèn phản hồi chính xác trong vòng 1-2 giây.

Lệnh Đổi màu: *"Ok Google, set the LED to Red/Blue/Green"* -> Google Assistant nhận diện giọng nói và truyền lệnh xuống ESP32 qua Sinric Pro. Màu sắc thay đổi đúng theo yêu cầu.

Lệnh Điều chỉnh độ sáng: *"Ok Google, set LED brightness to 50%"* -> Hệ thống giảm cường độ sáng chính xác theo tỷ lệ phần trăm.

Nhận xét: Độ trễ khi điều khiển bằng giọng nói (khoảng 1.5 - 2.5 giây) cao hơn so với điều khiển qua Web nội bộ. Nguyên nhân là do lệnh phải đi qua máy chủ của

Google và Sinric Pro trên Cloud rồi mới quay về thiết bị. Tuy nhiên, mức trễ này vẫn nằm trong phạm vi chấp nhận được và tạo ra trải nghiệm người dùng rất tốt.

4.4. Đánh giá ưu và nhược điểm của hệ thống

Sau thời gian vận hành thực nghiệm, hệ thống bộc lộ những ưu điểm vượt trội và một số hạn chế cần khắc phục như sau:

4.4.1. Ưu điểm

Giá thành rẻ: So với các hệ thống đèn thông minh thương mại như Philips Hue (có giá hàng triệu đồng), giải pháp tự xây dựng với ESP32 chỉ tiêu tốn một phần nhỏ chi phí nhưng tính năng tương đương.

Khả năng tùy biến cực cao: Người dùng có thể tự viết thêm các hiệu ứng ánh sáng mới, thay đổi giao diện Web hoặc tích hợp thêm các cảm biến khác vào hệ thống một cách dễ dàng.

Tính đa nền tảng: Hệ thống hoạt động mượt mà trên cả trình duyệt web (không cần cài app) và ứng dụng nhà thông minh chuyên dụng (Google Home).

Hiệu năng ổn định: ESP32 hoạt động bền bỉ, không bị nóng hay treo mạch khi chạy liên tục trong nhiều giờ nhờ thiết kế phân cứng có tản nhiệt và tụ lọc nguồn đầy đủ.

4.4.2. Nhược điểm và hạn chế

Phụ thuộc vào kết nối Wi-Fi: Đây là nhược điểm cố hữu của các thiết bị IoT. Nếu Router Wi-Fi gặp sự cố hoặc mất kết nối Internet, tính năng điều khiển qua giọng nói và điều khiển từ xa qua Cloud sẽ bị vô hiệu hóa hoàn toàn.

Vấn đề cấp nguồn cho dải LED dài: Với các dải LED dài hơn (trên 5m), hiện tượng sụt áp ở cuối dải vẫn có thể xảy ra nếu không được cấp nguồn bổ sung tại nhiều điểm dọc theo dải LED.

Độ bảo mật: Giao diện Web nội bộ hiện tại chưa có lớp xác thực (Login), bất kỳ ai trong cùng mạng Wi-Fi đều có thể truy cập IP và điều khiển đèn. Điều này cần được cải thiện bằng cách thêm trang đăng nhập trong các phiên bản nâng cấp sau.

5. PHÀN KẾT LUẬN VÀ KIẾN NGHỊ (HƯỚNG PHÁT TRIỂN)

5.1. Kết luận chung

Sau một thời gian nghiên cứu lý thuyết nghiêm túc, tiến hành thiết kế linh kiện và lập trình thực nghiệm, đề tài “**Phát triển hệ thống đèn LED thông minh RGB với ESP32: Điều khiển qua ứng dụng web và giọng nói**” đã hoàn thành xuất sắc các mục tiêu ban đầu đề ra. Bức tranh toàn cảnh về một hệ thống IoT ứng dụng trong không gian nhà thông minh (Smart Home) đã được mô phỏng và hiện thực hóa thành công thông qua mô hình thực tế.

Cụ thể, đề tài đã đạt được những kết quả nổi bật sau:

Về mặt phần cứng: Hệ thống hoạt động cực kỳ ổn định. Việc tính toán và sử dụng nguồn điện công suất cao kết hợp với các linh kiện bảo vệ (điện trở, tụ điện lọc nguồn) đã giúp dải LED WS2812 phát huy tối đa khả năng hiển thị dải màu RGB chân thực mà không xảy ra hiện tượng sput áp hay nhiễu tín hiệu.

Về mặt phần mềm nội bộ: Mã nguồn (Firmware) nạp trên ESP32 được tối ưu hóa bằng phương pháp lập trình bất đồng bộ (non-blocking). Giao diện Web Server nội bộ được thiết kế trực quan, hiện đại, tương thích tốt trên nhiều kích thước màn hình thiết bị di động. Các thao tác đổi màu, thay đổi độ sáng hay kích hoạt hiệu ứng động (Rainbow, Pulse, Fade) đều phản hồi chính xác với độ trễ vô cùng thấp (dưới 150ms).

Về tích hợp hệ sinh thái đám mây: Dự án đã thiết lập thành công cầu nối giao tiếp MQTT thông qua nền tảng trung gian Sinric Pro. Từ đó, cấp quyền điều khiển thiết bị cho trợ lý ảo Google Assistant. Các câu lệnh bằng giọng nói được hệ thống nhận diện tốt, xử lý mượt mà, mang lại trải nghiệm tiện nghi, rảnh tay

(hands-free) đúng với tiêu chuẩn của các thiết bị nhà thông minh thương mại hiện nay.

Tóm lại, hệ thống không chỉ mang giá trị học thuật trong việc làm chủ công nghệ vi điều khiển cốt lõi, mà còn chứng minh được tính khả thi trong việc tự xây dựng các thiết bị IoT với chi phí thấp nhưng vẫn đảm bảo hiệu năng và tính thẩm mỹ.

5.2. Kiến nghị và Hướng phát triển tương lai

Mặc dù mô hình hiện tại đã đáp ứng tốt các yêu cầu cơ bản của một hệ thống chiếu sáng thông minh, nhưng để tiệm cận với một sản phẩm thương mại hoàn chỉnh và nâng cao hơn nữa trải nghiệm người dùng, hệ thống có thể được nghiên cứu và mở rộng theo các hướng sau:

5.2.1. Phát triển phần cứng chuyên nghiệp (Thiết kế PCB) Hiện tại, mô hình thử nghiệm đang sử dụng mạch phát triển (DevKit) kết nối thông qua các dây cắm (jumper) hoặc hàn trên bảng mạch đục lỗ (perfboard). Điều này có thể dẫn đến rủi ro lỏng lẻo vật lý, oxy hóa mối nối khi hoạt động trong thời gian dài. Hướng phát triển thiết yếu là sử dụng các phần mềm thiết kế mạch điện tử (như Altium Designer hoặc KiCad) để vẽ và đặt gia công mạch in (PCB) chuyên nghiệp. Việc thiết kế PCB riêng không chỉ giúp thu nhỏ tối đa kích thước hộp điều khiển, tăng khả năng tản nhiệt cho chip ESP32 mà còn cho phép tích hợp các giắc cắm công nghiệp (Terminal Block), giúp quá trình lắp đặt tại nhà người dùng trở nên an toàn và dễ dàng hơn.

5.2.2. Bổ sung tính năng đồng bộ ánh sáng theo âm nhạc (Music Sync) Để tăng tính giải trí cho không gian (ví dụ: phòng xem phim, phòng gaming), hệ thống có thể được tích hợp thêm module cảm biến âm thanh (như micro analog MAX4466 hoặc micro kỹ thuật số giao tiếp chuẩn I2S như INMP441). Bằng cách thu thập tín hiệu âm thanh từ môi trường xung quanh và áp dụng thuật toán Biến đổi Fourier nhanh (FFT - Fast Fourier Transform) trực tiếp trên nhân xử lý của ESP32, hệ thống có thể phân tách các dải tần số (Bass, Mid, Treble). Từ đó, điều khiển dải LED WS2812 chớp nháy, đổi màu và thay đổi biên độ sáng đồng bộ hoàn toàn với nhịp điệu bài hát, tạo ra một không gian ánh sáng sống động.

5.2.3. Khắc phục sự phụ thuộc mạng và nâng cấp bảo mật Như đã phân tích ở Chương 4, hệ thống hiện tại phụ thuộc hoàn toàn vào mạng Wi-Fi và Internet để sử dụng tính năng giọng nói. Để đảm bảo tính liên tục (Local Fallback), phiên bản phần mềm tiếp theo nên tích hợp thêm giao thức Bluetooth Low Energy (BLE) có sẵn trên lõi ESP32. Nếu Wi-Fi mất kết nối, người dùng vẫn có thể kết nối điện thoại trực tiếp với bộ điều khiển qua Bluetooth để thao tác. Ngoài ra, giao diện Web cục bộ cần được bổ sung thêm một lớp bảo mật xác thực (yêu cầu nhập Tên đăng nhập và Mật khẩu) trước khi hiển thị bảng điều khiển, nhằm ngăn chặn các truy cập trái phép từ những thiết bị lạ trong cùng mạng nội bộ.

5.2.4. Cập nhật giao thức Matter Trong tương lai xa, để thiết bị không bị bó buộc vào riêng hệ sinh thái của Google, mã nguồn có thể được nâng cấp để hỗ trợ chuẩn giao tiếp nhà thông minh thế hệ mới: **Matter**. Việc hỗ trợ Matter sẽ giúp đèn LED này có thể hoạt động đồng thời và mượt mà trên cả Google Home, Apple HomeKit và Amazon Alexa mà không cần phải phụ thuộc vào các máy chủ đám mây của bên thứ ba như hiện tại.

TÀI LIỆU THAM KHẢO

I. Tài liệu tiếng Việt (*Sắp xếp theo thứ tự bảng chữ cái ABC dựa trên Tên tác giả*)

[1] Phạm Quang Huy (2021). *Lập trình vi điều khiển ESP32*, Nhà xuất bản Bách Khoa, Hà Nội.

[2] Lê Văn Thắng (2020). "Thiết kế hệ thống điều khiển nhà thông minh", *Tạp chí Khoa học và Công nghệ*, tập 5, số 2, tr. 15-20.

II. Tài liệu tiếng Anh (*Sắp xếp theo thứ tự bảng chữ cái ABC dựa trên Họ tác giả*)

[3] Kolban, Neil (2018). *Kolban's Book on ESP32*, Leanpub.

[4] Teckel, T. (2019). "Understanding the WS2812 protocol", *Journal of Open Source Hardware*, vol. 2, pp. 40-45.

7. PHỤ LỤC

Phụ lục 1: Mã nguồn C++ điều khiển trung tâm ESP32 (File main.ino)

Dưới đây là toàn bộ mã nguồn C++ được biên dịch và nạp vào vi điều khiển ESP32 thông qua môi trường Arduino IDE.

```
Main.cpp
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_NeoPixel.h>
#include <ArduinoJson.h>
#include <SPIFFS.h>

// --- 1. Cấu hình Kết nối WiFi ---
const char* ssid = "Wokwi-GUEST"; // Wokwi mặc định
const char* password = ""; // Không cần password

// --- 2. Cấu hình LED RGB (WS2812) ---
#define LED_PIN 5
#define NUM_LEDS 60 // Số lượng LED WS2812
#define BRIGHTNESS 255

Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);

// --- 3. Web Server ---
WebServer server(80);

// --- 4. Biến điều khiển LED ---
struct LEDState {
    uint8_t r = 255;
    uint8_t g = 0;
```

```

uint8_t b = 0;
uint8_t brightness = 255;
int mode = 0; // 0=solid, 1=rainbow, 2=pulse, 3=fade
int speed = 50;
} ledState;

```

```

enum Mode { SOLID = 0, RAINBOW = 1, PULSE = 2, FADE = 3, STROBE = 4 };
Mode currentMode = SOLID;

```

```

unsigned long lastUpdate = 0;
static uint16_t offset = 0;

```

```

// --- 5. Utility Functions ---
uint32_t Wheel(byte WheelPos) {
    WheelPos = 255 - WheelPos;
    if (WheelPos < 85) {
        return strip.Color(255 - WheelPos * 3, 0, WheelPos * 3);
    } else if (WheelPos < 170) {
        WheelPos -= 85;
        return strip.Color(0, WheelPos * 3, 255 - WheelPos * 3);
    } else {
        WheelPos -= 170;
        return strip.Color(WheelPos * 3, 255 - WheelPos * 3, 0);
    }
}

```

```

void setSolidColor(uint8_t r, uint8_t g, uint8_t b) {
    for (int i = 0; i < NUM_LEDS; i++) {
        strip.setPixelColor(i, strip.Color(r, g, b));
    }
    strip.show();
}

```

```

void rainbowEffect() {
  for (int i = 0; i < NUM_LEDS; i++) {
    uint8_t hue = (i * 256 / NUM_LEDS + offset) & 255;
    strip.setPixelColor(i, Wheel(hue));
  }
  strip.show();
  offset++;
}

```

```

void pulseEffect() {
  uint8_t brightness = (sin(millis() / 500.0) + 1) * 127;
  for (int i = 0; i < NUM_LEDS; i++) {
    strip.setPixelColor(i, strip.Color(
      (ledState.r * brightness) / 255,
      (ledState.g * brightness) / 255,
      (ledState.b * brightness) / 255
    ));
  }
  strip.show();
}

```

```

void fadeEffect() {
  uint8_t brightness = (sin(millis() / 1000.0) + 1) * 127;
  for (int i = 0; i < NUM_LEDS; i++) {
    strip.setPixelColor(i, strip.Color(
      (ledState.r * brightness) / 255,
      (ledState.g * brightness) / 255,
      (ledState.b * brightness) / 255
    ));
  }
  strip.show();
}

```

```
}
```

```
void strobeEffect() {  
  if ((millis() / 100) % 2 == 0) {  
    setSolidColor(ledState.r, ledState.g, ledState.b);  
  } else {  
    setSolidColor(0, 0, 0);  
  }  
}
```

```
// --- 6. HTTP Handlers ---
```

```
void handleRoot() {  
  String html = R"HTML(  
<!DOCTYPE html>  
<html>  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>LED RGB Control</title>  
<style>  
  * { margin: 0; padding: 0; box-sizing: border-box; }  
  body {  
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
    min-height: 100vh;  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    padding: 20px;  
  }  
  .container {  
    background: white;
```



```
border-radius: 20px;
box-shadow: 0 20px 60px rgba(0,0,0,0.3);
padding: 40px;
max-width: 500px;
width: 100%;
}
h1 {
  text-align: center;
  color: #333;
  margin-bottom: 30px;
  font-size: 28px;
}
.control-section {
  margin-bottom: 30px;
}
label {
  display: block;
  margin-bottom: 8px;
  color: #555;
  font-weight: 600;
  font-size: 14px;
}
input[type="color"] {
  width: 100%;
  height: 60px;
  border: none;
  border-radius: 10px;
  cursor: pointer;
}
input[type="range"] {
  width: 100%;
  height: 6px;
```

```
border-radius: 3px;
background: #ddd;
outline: none;
-webkit-appearance: none;
}
input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 20px;
  height: 20px;
  border-radius: 50%;
  background: #667eea;
  cursor: pointer;
}
input[type="range"]::-moz-range-thumb {
  width: 20px;
  height: 20px;
  border-radius: 50%;
  background: #667eea;
  cursor: pointer;
}
.value-display {
  text-align: center;
  color: #667eea;
  font-weight: bold;
  margin-top: 8px;
}
.mode-buttons {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
  margin-bottom: 20px;
```

```

}
button {
  padding: 12px 20px;
  border: none;
  border-radius: 10px;
  background: #667eea;
  color: white;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.3s;
  font-size: 14px;
}
button:hover {
  background: #764ba2;
  transform: translateY(-2px);
}
button.active {
  background: #764ba2;
  box-shadow: 0 5px 15px rgba(118, 75, 162, 0.4);
}
.spinner {
  display: inline-block;
  width: 20px;
  height: 20px;
  border: 3px solid rgba(255,255,255,.3);
  border-radius: 50%;
  border-top-color: white;
  animation: spin 1s ease-in-out infinite;
}
@keyframes spin {
  to { transform: rotate(360deg); }
}

```

```

.status {
  text-align: center;
  padding: 15px;
  background: #f0f0f0;
  border-radius: 10px;
  margin-top: 20px;
  font-size: 14px;
  color: #666;
}
.status.success {
  background: #d4edda;
  color: #155724;
}
</style>
</head>
<body>
<div class="container">
  <h1> LED RGB Control</h1>

  <div class="control-section">
    <label>Chế độ hiển thị</label>
    <div class="mode-buttons">
      <button onclick="setMode(0)" id="btn-solid">Solid Color</button>
      <button onclick="setMode(1)" id="btn-rainbow">Rainbow</button>
      <button onclick="setMode(2)" id="btn-pulse">Pulse</button>
      <button onclick="setMode(3)" id="btn-fade">Fade</button>
      <button onclick="setMode(4)" id="btn-strobe">Strobe</button>
      <button onclick="setMode(-1)" id="btn-off">Tắt</button>
    </div>
  </div>
</div>

```

```

<div class="control-section" id="color-section">

```

```

    <label>Chọn màu sắc</label>

    <input type="color" id="colorPicker" value="#ff0000"
onchange="updateColor()">
  </div>

```

```

<div class="control-section">
  <label>Độ sáng: <span id="brightnessValue">255</span></label>

  <input type="range" id="brightness" min="0" max="255" value="255"
oninput="updateBrightness()">
</div>

```

```

<div class="control-section" id="speed-section" style="display:none;">
  <label>Tốc độ: <span id="speedValue">50</span></label>

  <input type="range" id="speed" min="10" max="200" value="50"
oninput="updateSpeed()">
</div>

```

```

<div class="status" id="status">Ready...</div>
</div>

```

```

<script>
  const API_URL = '/api';

  function setMode(mode) {
    if (mode === -1) {
      fetch(API_URL + '/off', { method: 'POST' })
        .then(() => updateStatus('LED tắt', 'success'))
        .catch(() => updateStatus('Lỗi: Không thể kết nối'));
      return;
    }

    fetch(API_URL + '/mode', {

```

```

    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ mode: mode })
  })
  .then(() => {
    updateButtonStates();
    updateStatus('Thay đổi chế độ thành công', 'success');
    document.getElementById('color-section').style.display = mode === 0 ? 'block' :
'none';
    document.getElementById('speed-section').style.display = mode > 0 ? 'block' :
'none';
  })
  .catch(() => updateStatus('Lỗi: Không thể kết nối'));
}

```

```

function updateColor() {
  const color = document.getElementById('colorPicker').value;
  const r = parseInt(color.substr(1,2), 16);
  const g = parseInt(color.substr(3,2), 16);
  const b = parseInt(color.substr(5,2), 16);

  fetch(API_URL + '/color', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ r: r, g: g, b: b })
  })
  .then(() => updateStatus('Màu sắc được cập nhật', 'success'))
  .catch(() => updateStatus('Lỗi: Không thể kết nối'));
}

```

```

function updateBrightness() {
  const brightness = document.getElementById('brightness').value;

```

```

document.getElementById('brightnessValue').textContent = brightness;

fetch(API_URL + '/brightness', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ brightness: parseInt(brightness) })
})
.catch(() => updateStatus('Lỗi: Không thể kết nối'));
}

```

```

function updateSpeed() {
  const speed = document.getElementById('speed').value;
  document.getElementById('speedValue').textContent = speed;

  fetch(API_URL + '/speed', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ speed: parseInt(speed) })
  })
  .catch(() => updateStatus('Lỗi: Không thể kết nối'));
}

```

```

function updateButtonStates() {
  document.querySelectorAll('.mode-buttons button').forEach(btn =>
btn.classList.remove('active'));
  document.getElementById('btn-solid').classList.add('active');
}

```

```

function updateStatus(msg, type = 'info') {
  const status = document.getElementById('status');
  status.textContent = msg;
  status.className = 'status ' + type;
}

```

```
}
```

```
    updateButtonStates();  
</script>  
</body>  
</html>  
)HTML";  
  
server.send(200, "text/html", html);  
}
```

```
void handleAPIColor() {  
    if (server.method() == HTTP_POST) {  
        String body = server.arg("plain");  
        StaticJsonDocument<256> doc;  
        deserializeJson(doc, body);  
  
        ledState.r = doc["r"];  
        ledState.g = doc["g"];  
        ledState.b = doc["b"];  
        currentMode = SOLID;  
  
        server.send(200, "application/json", "{\"status\":\"ok\"}");  
    }  
}
```

```
void handleAPIBrightness() {  
    if (server.method() == HTTP_POST) {  
        String body = server.arg("plain");  
        StaticJsonDocument<256> doc;  
        deserializeJson(doc, body);
```



```

    ledState.brightness = doc["brightness"];
    strip.setBrightness(ledState.brightness);

    server.send(200, "application/json", "{\"status\":\"ok\"}");
}
}

```

```

void handleAPIMode() {
    if (server.method() == HTTP_POST) {
        String body = server.arg("plain");
        StaticJsonDocument<256> doc;
        deserializeJson(doc, body);

        currentMode = (Mode)doc["mode"];

        server.send(200, "application/json", "{\"status\":\"ok\"}");
    }
}

```

```

void handleAPIOff() {
    if (server.method() == HTTP_POST) {
        strip.clear();
        strip.show();

        server.send(200, "application/json", "{\"status\":\"ok\"}");
    }
}

```

```

void handleAPISpeed() {
    if (server.method() == HTTP_POST) {
        String body = server.arg("plain");
        StaticJsonDocument<256> doc;

```

```

deserializeJson(doc, body);

ledState.speed = doc["speed"];

server.send(200, "application/json", "{\"status\":\"ok\"}");
}
}

```

```

void handleAPIStatus() {
  DynamicJsonDocument doc(512);
  doc["r"] = ledState.r;
  doc["g"] = ledState.g;
  doc["b"] = ledState.b;
  doc["brightness"] = ledState.brightness;
  doc["mode"] = currentMode;
  doc["speed"] = ledState.speed;

  String response;
  serializeJson(doc, response);
  server.send(200, "application/json", response);
}

```

```

// --- 7. Setup ---
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("\n\nStarting LED RGB Control System...");

  // Khởi tạo LED strip
  strip.begin();
  strip.show();
}

```

```

strip.setBrightness(ledState.brightness);

// Kết nối WiFi
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 30) {
    delay(500);
    Serial.print(".");
    attempts++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi connected!");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
} else {
    Serial.println("\nFailed to connect to WiFi");
}

```

```

// Setup HTTP routes
server.on("/", handleRoot);
server.on("/api/color", handleAPIColor);
server.on("/api/brightness", handleAPIBrightness);
server.on("/api/mode", handleAPIMode);
server.on("/api/off", handleAPIOff);
server.on("/api/speed", handleAPISpeed);
server.on("/api/status", handleAPIStatus);

server.begin();
Serial.println("HTTP server started");

```

```
// Hiệu ứng khởi động
setSolidColor(0, 255, 0);
delay(500);
setSolidColor(0, 0, 0);
}
```

```
// --- 8. Loop ---
void loop() {
  server.handleClient();

  // Cập nhật LED theo mode
  if (millis() - lastUpdate > (260 - ledState.speed)) {
    switch (currentMode) {
      case SOLID:
        setSolidColor(ledState.r, ledState.g, ledState.b);
        break;
      case RAINBOW:
        rainbowEffect();
        break;
      case PULSE:
        pulseEffect();
        break;
      case FADE:
        fadeEffect();
        break;
      case STROBE:
        strobeEffect();
        break;
    }
    lastUpdate = millis();
  }
}
```

```
}
```

Phụ lục 2: Mã nguồn Diagram.json

```
{
  "version": 1,
  "author": "Nho",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": -57.6, "left": -71.96, "attrs": {} },
    {
      "type": "wokwi-led-matrix",
      "id": "matrix1",
      "top": -54,
      "left": 63,
      "attrs": { "rows": "8", "cols": "8", "brightness": "6" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "esp:GND.2", "matrix1:VSS", "black", [ "v0", "h9.6", "v192", "h124.8" ] ],
    [ "matrix1:VDD", "esp:5V", "red", [ "v28.8", "h-230.8", "v-28.8" ] ],
    [ "matrix1:DIN", "esp:2", "green", [ "v9.6", "h-126", "v-48" ] ]
  ],
  "dependencies": {}
}
```